

SovereignAI Workbench — API Reference

SIH 2026 · PS 26117 · Team Blunder · for the frontend team

Base URL	http://192.168.59.10:8080 (ask Himanshu if the VM IP changes)
Live API docs	GET /docs — Swagger UI. Try any endpoint in the browser first.
CORS	Open to all origins. A React dev server on any port works.
Auth	None. Single-tenant, air-gapped system.
Content-Type	application/json for all POST bodies.

1. GET /api/health

Poll on mount and every 30s. Drives the status bar.

```
{
  "status": "ok", // "ok" | "degraded"
  "inference_endpoint": "http://10.0.2.2:11434",
  "inference_reachable": true,
  "models": [ {"id": "general", "name": "qwen3.5:2b-q4_K_M",
    "role": "general", "present": true} ],
  "sandbox_available": true,
  "tools": ["read_document", "list_files", "run_python", ...],
  "timestamp": "2026-09-13T02:40:11"
}
```

degraded = Ollama unreachable. Show a red banner; nothing else will work.

2. GET /api/models

Model side panel. **resident** = currently loaded in GPU VRAM (highlight these).

```
{
  "registry_file": "/app/models/registry.yaml",
  "fallback": "general",
  "models": [{
    "id": "general", "name": "qwen3.5:2b-q4_K_M", "role": "general",
    "capabilities": ["text", "reasoning", "tools", "vision", "orchestrator"],
    "size_gb": 1.6, "description": "Document summarisation, drafting...",
    "options": {"temperature": 0.3, "num_ctx": 16384},
    "resident": true, "vram_gb": 1.75
  }]
}
```

3. POST /api/route

Routing decision only, no model call in the common case. Use it to preview which model will handle the task before the user hits Run.

REQUEST

```
{"task": "Draft an approval note", "attachments": []}
```

RESPONSE

```
{
  "task_type": "document", // chat | document | code | vision | embedding
  "model_id": "general",
  "model_name": "qwen3.5:2b-q4_K_M",
  "reason": "document signals score 8.0 (3 matched)",
  "method": "deterministic", // "deterministic" 0-1ms | "llm" ~400ms
  "confidence": 1.0,
  "routing_latency_ms": 1
}
```

4. POST /api/task/stream — PRIMARY ENDPOINT

Server-sent events, one per agent step. Use this rather than /api/task so the user sees progress instead of a 7–25 second spinner.

REQUEST

```
{ "task": "Read data/samples/inspection_report.png and summarise it",  
  "attachments": [] }
```

EVENT STREAM — each event is data: {json}\n\n — switch on .type

```
// 1. routing (first event, same fields as /api/route + "type")  
  
// 2. step (one per agent action, arrives as it completes)  
{ "type": "step", "n": 1, "kind": "tool", // "tool" | "final"  
  "tool": "read_document", "ok": true,  
  "arguments": { "path": "data/samples/inspection_report.png",  
    "observation": "MANGALORE REFINERY - EQUIPMENT INSPECTION...",  
    "content": "", "elapsed_ms": 1105, "model": "qwen3.5:2b-q4_K_M" }  
  
// kind="final" -> "content" holds the answer, "tool" is empty  
  
// 3. done (last event)  
{ "type": "done", "steps": 3,  
  "tool_calls": [ "read_document", "create_word_document",  
    "orchestrator_model": "qwen3.5:2b-q4_K_M",  
    "delegate_models": [], "tools_offered": [ "read_document", ... ],  
    "artifacts": [ "outputs/approval_note.docx",  
    "total_ms": 7025, "stopped_reason": "model returned a final answer",  
    "answer": "I have created the approval note..." }  
  
// 4. error  
{ "type": "error", "error": "TypeError: ..." }
```

REACT: EventSource cannot POST a body. Use fetch + a stream reader.

```
const res = await fetch(`${API}/api/task/stream`, {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({ task, attachments: [] })  
});  
  
const reader = res.body.getReader();  
const decoder = new TextDecoder();  
let buffer = "";  
  
while (true) {  
  const { value, done } = await reader.read();  
  if (done) break;  
  buffer += decoder.decode(value, { stream: true });  
  const parts = buffer.split("\n\n");  
  buffer = parts.pop(); // <-- keep the incomplete tail  
  for (const p of parts) {  
    if (!p.startsWith("data: ")) continue;  
    handleEvent(JSON.parse(p.slice(6)));  
  }  
}
```

The **buffer = parts.pop()** line is not optional — events split across chunks.

5. POST /api/workflow – the flagship demo

Four fixed stages, ~12s, far more reliable than the open agent. Not streamed: show a 4-step progress indicator and await the response.

```
REQUEST
{ "workflow": "inspection_to_approval",
  "document_path": "data/samples/inspection_report.png",
  "output_filename": "approval_note.docx" }

RESPONSE
{ "workflow": "inspection_to_approval",
  "stages": [
    { "n": 1, "name": "ingest document", "detail": "ocr in 321 ms, 873 chars",
      "ok": true, "elapsed_ms": 321 },
    { "n": 2, "name": "extract structured fields",
      "detail": "P-101A | HIGH | 4 findings", "ok": true, "elapsed_ms": 2636 },
    { "n": 3, "name": "consult SOPs", "detail": "top 0.801 [SOP-MECH-014...]",
      "ok": true, "elapsed_ms": 7958 },
    { "n": 4, "name": "draft approval note", "detail": "Created Word document...",
      "ok": true, "elapsed_ms": 1898 } ],
  "artifacts": [ "outputs/approval_note.docx" ],
  "summary": "Processed inspection_report.png: P-101A (HIGH)...",
  "total_ms": 12821, "failed": "" }
```

GET /api/workflows lists the available workflows.

6. Knowledge base

```
GET /api/kb/status -> { "total_chunks": 20, "documents": [...],
  "embed_model": "nomic-embed-text" }
POST /api/kb/reindex -> re-reads data/kb/, returns the new stats
POST /api/kb/ask body: { "task": "what is the seal lead time" }

{ "question": "...", "confident": true, "top_score": 0.820,
  "answer": "The lead time for a standard cartridge seal is 21 days
  from purchase order [CORR-2026-0318... #3].",
  "sources": [ { "citation": "[CORR-2026-0318... #3]", "score": 0.82,
    "source": "data/kb/CORR-...md", "excerpt": "..."} ],
  "elapsed_ms": 7362, "model": "qwen3.5:2b-q4_K_M" }
```

confident:false means retrieval scored below the 0.58 floor and no model was called. Render those answers in grey with a 'low confidence' tag — do not present them as organisational requirements. Always render the sources list; the citations are the point.

7. Files

```
GET /api/files
{ "samples":[{"name":"inspection_report.png",
"path":"data/samples/inspection_report.png",
"size":68761,"area":"sample","modified":"2026-09-12T23:17:32"}],
"uploads":[...], "outputs":[...] }
```

```
POST /api/upload multipart/form-data, field name: "file"
-> {"ok":true,"name":"x.pdf","path":"data/uploads/x.pdf","size":12345}
```

```
GET /api/download/{filename}
-> the file itself. Use for generated .docx. Plain <a href> works.
```

Use **path** from `/api/files` as the **document_path** for a workflow, or put it in **attachments** for a task.

8. Sovereignty monitor

```
GET /api/monitor/egress
{ "total_requests":10, "external_count":0, "sovereign":true,
"by_classification":{"LOCAL":0,"PRIVATE":10,"EXTERNAL":0},
"destinations":["10.0.2.2:11434"],
"events":[{"ts":"...", "host":"10.0.2.2", "port":11434,
"classification":"PRIVATE", "status":200, "elapsed_ms":4225}],
"report":"...pre-formatted text block..." }
```

```
POST /api/monitor/egress/reset clear the log before a demo run
POST /api/monitor/capture/start begin tcpdump capture
POST /api/monitor/capture/stop -> summary + "report" text
GET /api/monitor/classify?host=8.8.8.8
```

Poll **/api/monitor/egress** every 5–6s. This panel is the single most important thing on screen for judging: a persistent badge reading **EGRESS — 0 EXTERNAL** in green, flipping to red if `external_count > 0`. classification is LOCAL / PRIVATE / EXTERNAL.

Suggested screens

Workbench (main)	Task input + preset chips. Live step trace from the SSE stream. Download button for each artifact. This is where the demo happens.
Sovereignty	Egress panel, destination list, classification counts, big green/red verdict. Judges will look at this.
Knowledge base	Ask a question, show the answer with its citation chips and scores. Include one question the SOPs cannot answer, to show the refusal.
Models	Registry cards, VRAM residency, and the routing preview from <code>/api/route</code> .

Timings to design around

Routing	0–1 ms deterministic, ~400 ms when the LLM classifier runs
Simple task	1–6 s, usually 2 steps
Flagship workflow	~12 s, 4 stages
Agent with tools	7–25 s, 2–4 steps
KB question	5–9 s

Nothing here is instant. Every action needs a visible progress state, and the streaming endpoint exists so the user watches real steps rather than a spinner.

Repo: github.com/HimanshuGit3/SovereignAI-Hackathon · A working reference UI is at web/index.html — plain HTML, no build step. Read it for the stream-parsing logic, then build the real thing properly in React.